マイコンシステム
第１５回

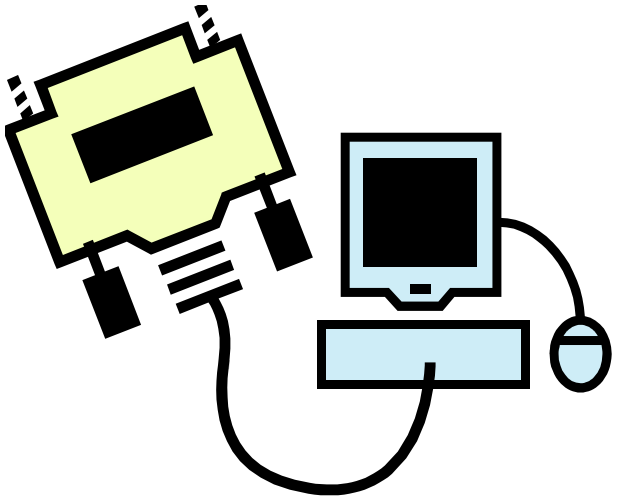
青森大学 ソフトウェア情報学部

橋本恭能

haship@aomori-u.ac.jp

目次

- 前々回課題の解答例
- 講義
  - テスト
- 例題
  - 仕様変更①
- 課題
  - 仕様変更②



前々回課題の解答例

時間設定中から遷移した後

時間と状態はどう表示されるか？

課題の解答例

- 解答

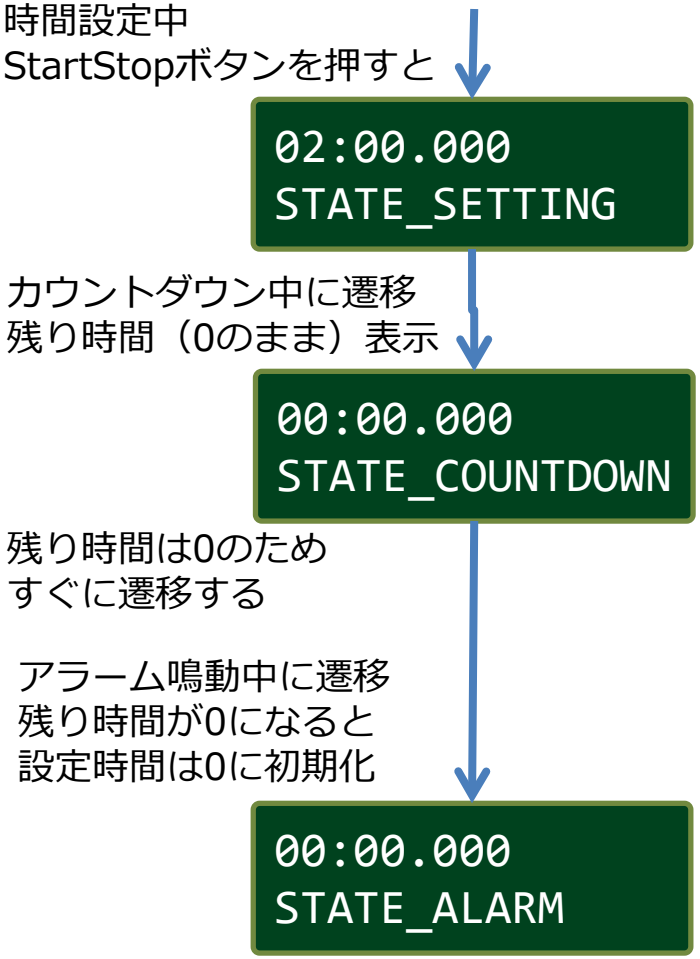
  2分以外でも可

  - 時間：02.00.000 -> 00:00.000
  - 状態：STATE_SETTING

    (-> STATE_COUNTDOWN を経由して)

    -> STATE_ALARM

- なぜ状態がアラーム鳴動中に？
  - 時間設定中からカウントダウン中に遷移した後、残り時間の初期値が0のままになっているため、すぐにアラーム鳴動中に遷移した



講義

テスト

仕様変更

組込みシステム開発

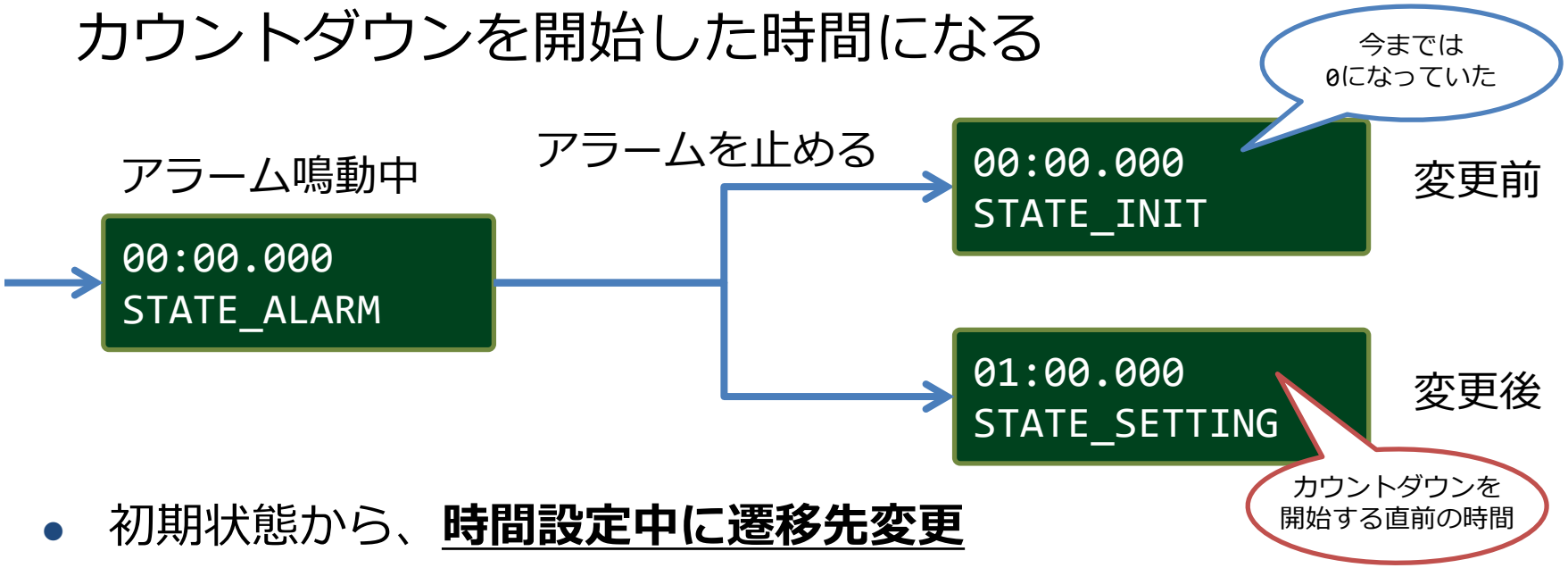
週	テーマ	内容
7	キッチンタイマーの組立	キッチンタイマーのハードを製作・確認
8 9	要求分析・外部設計	キッチンタイマーに必要な機能を抽出 画面表示、ボタンを押した時の動作を設定 機能の実現手順をフローチャートで図示
10 11 12	内部設計	状態遷移図・状態遷移表の使い方 システムの内部構造を考える 機能の実現手順をより詳細に決める
13 14	実装	プログラムを作る
15	テスト・完成	設計通りにできているか動作検証する

例題
仕様変更①

アラームを止めた後
カウントダウンを開始した時の
設定時間を表示する

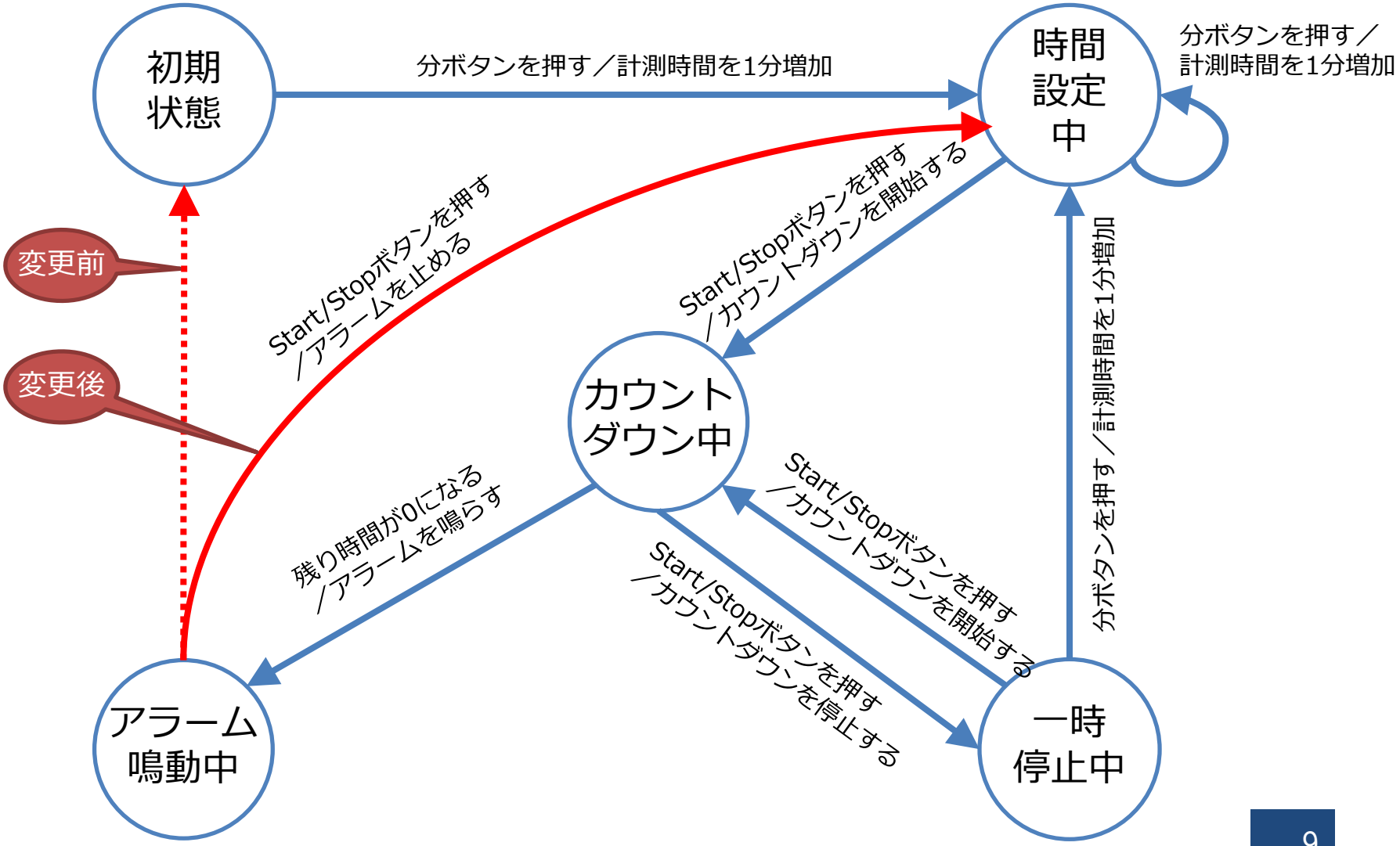
仕様変更①

- アラーム鳴動中にStart/Stopボタンを押して
  アラームを止めると、計測時間は
  カウントダウンを開始した時間になる



- 初期状態から、**時間設定中に遷移先変更**
- 設定時間**timeを保存**しておく必要あり
- それに伴う必要な変更を行う

状態遷移図の変更


初期
状態
分ボタンを押す／計測時間を1分増加
時間
設定
中
分ボタンを押す／
計測時間を1分増加
変更前
変更後
Start/Stopボタンを押す
／アラームを止める
Start/Stopボタンを押す
／カウントダウンを開始する
カウント
ダウン中
残り時間が0になる
／アラームを鳴らす
Start/Stopボタンを押す
／カウントダウンを開始する
Start/Stopボタンを押す
／カウントダウンを停止する
分ボタンを押す／計測時間を1分増加
アラーム
鳴動中
一時
停止中

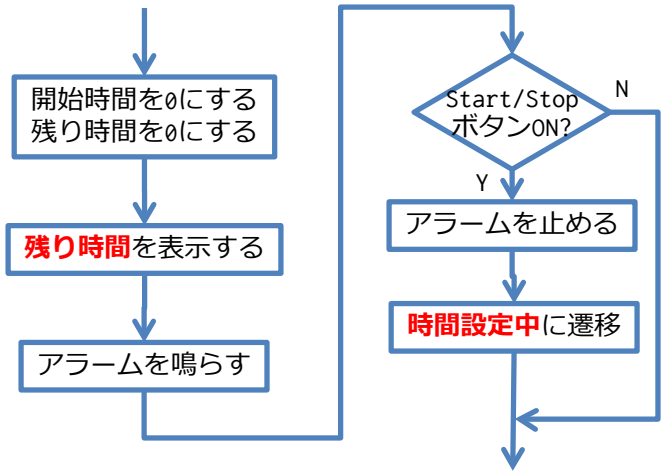
状態遷移表の変更

	初期状態	時間設定中	カウントダウン中	一時停止中	アラーム鳴動中
分ボタンを押す	計測時間を1分増加	計測時間を1分増加	／（無視）	計測時間を1分増加	／（無視）
	時間設定中	時間設定中		時間設定中	
Start/Stopボタンを押す	／（無視）	カウントダウンを開始する	カウントダウンを停止する	カウントダウンを開始する	アラームを止める
		カウントダウン中	一時停止中	カウントダウン中	**時間設定中**
残り時間が0になる	／（無視）	／（無視）	アラームを鳴らす	×（不可）	／（無視）
			アラーム鳴動中		

遷移先が変更

変更：アラーム鳴動中の処理内容

1. 必ず実行する処理
   - 残り時間を0にする
     （設定時間を0にしない）
   - **残り時間を表示する**
   - アラームを鳴らす
2. イベント発生時の処理
   - Start/Stopボタンを押したら
     アラームを止める、
     時間設定中に遷移




```
//アラーム鳴動中の処理（Stateタブ）
  case STATE_ALARM:
  // 今の状態で必ず または継続して実行する処理
  //time = 0L;
  remainTimer = 0L;
  displayTimer(remainTime, s);
  startAlarm();

  //イベント発生時の処理
  //Start/Stopボタンを押した場合
  if (e == EVENT_PUSH_START_STOP_BUTTON) {
    //アラームを止める
    stopAlarm();
    //今の状態を出るときに1回だけ実行する処理
      //なし
    //状態遷移
    ret = STATE_SETTING; //時間設定中に遷移
    //次の状態に入るとき1回だけ実行する処理
      //なし
  }
  break;
```

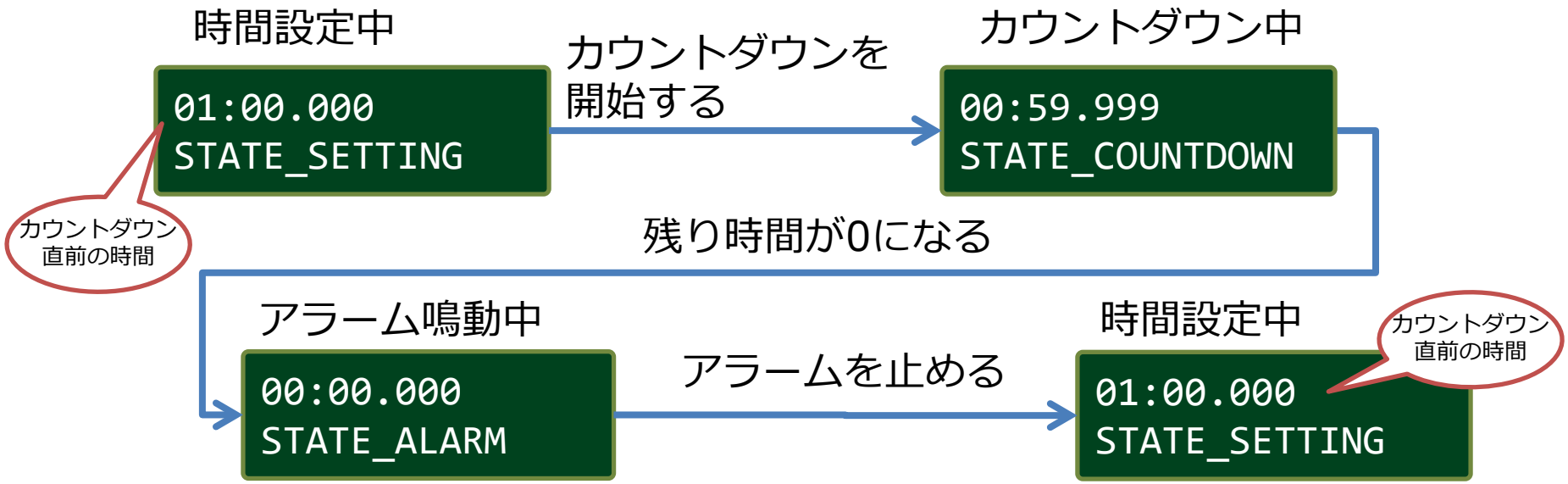
削除

変更

変更

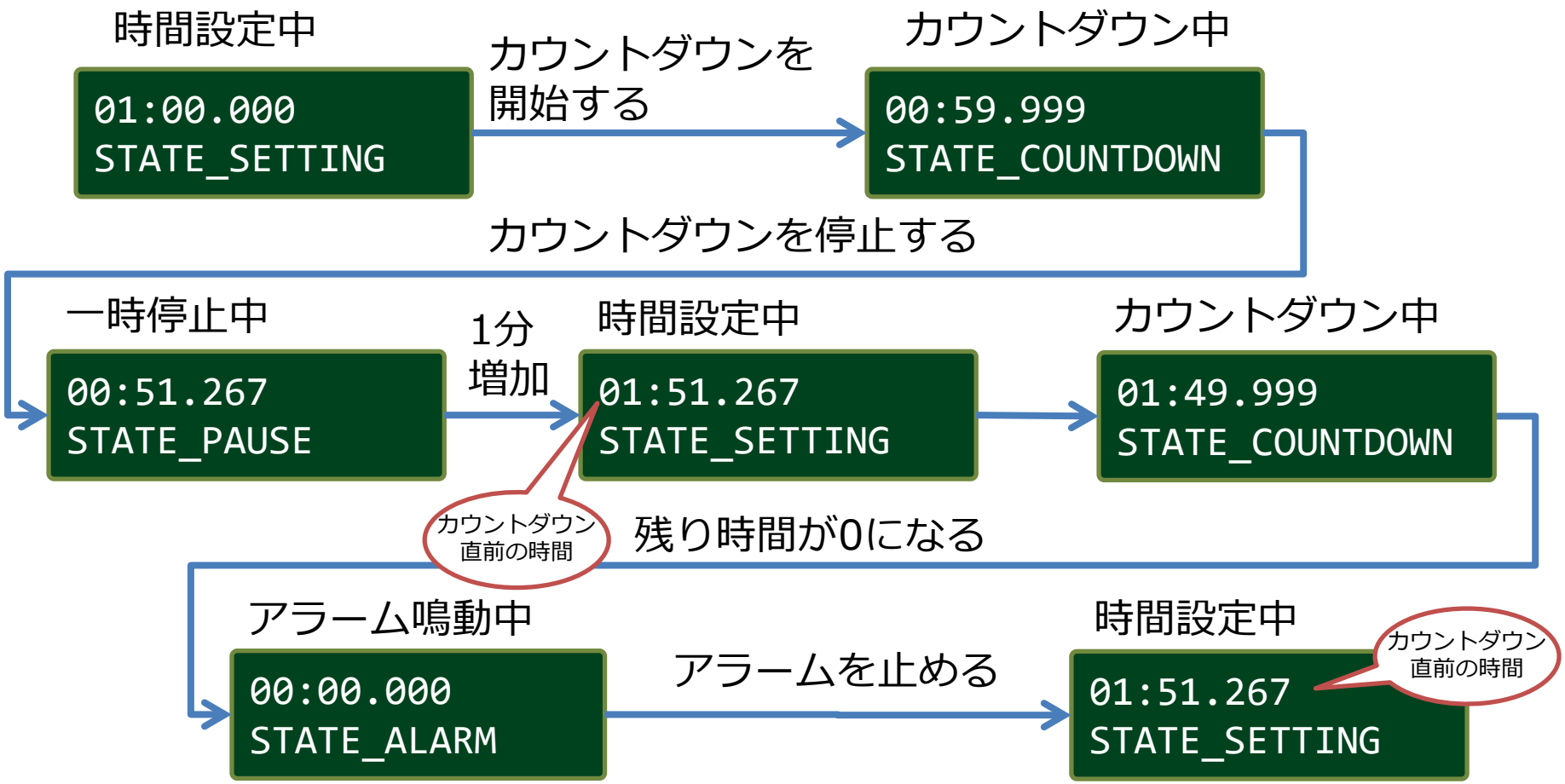
動作確認①

- 設定時間1分でカウントダウン開始
  - カウントダウン終了後、アラームを止めたら時間1分、状態：時間設定中になるか確認



動作確認②

- 一時停止して設定時間を増やす



例題完了時のプログラム

```
//config.h

//列挙体 状態
enum State {
  STATE_INIT,      //初期状態
  STATE_SETTING,   //設定中
  STATE_COUNTDOWN, //カウントダウン中
  STATE_PAUSE,     //一時停止中
  STATE_ALARM      //アラーム鳴動中
} state;

//列挙体 イベント
enum Event {
  EVENT_NONE,                    //イベントが発生しない
//ここに課題プログラムで追加
  EVENT_PUSH_MINUTE_BUTTON,      //分ボタンを押す
  EVENT_PUSH_START_STOP_BUTTON, //StartStopボタンを押す
  EVENT_TIME_IS_UP               //残り時間が0になる
} event;
```

メインタブ

```
#include ”config.h”
long time;          //長整数型　設定時間
long startTime;     //長整数型　カウントダウン開始時間
long remainTime;    //長整数型　残り時間

void setup() {
  setupDevice();          //デバイスの初期設定を行う
  time = 0L;              //設定時間を初期化する
  startTime = 0L;         //カウントダウン開始時間を初期化する
  remainTime = 0L;        //残り時間を初期化する
  state = STATE_INIT;     //状態変数を初期状態に設定する
  event = EVENT_NONE;     //イベント変数を"発生していない"に設定する
}

void loop() {
  //現在発生しているイベントを返す
  event = receiveEvent();
  //現在の状態とイベントから処理を実行し遷移する状態を返す
  state = handleEvent(state, event);
}
```

Actionタブ①

```
//Actionタブ

//設定時間を１分増加する
void addOneMinute(){
  time += 60000L;            //設定時間を60000ミリ秒加算する
}

//カウントダウンを開始する
void startTimer(){
  remainTime = time;         //残り時間に設定時間を代入
  startTime = millis();      //カウントダウン開始時間に現在の時間を代入
}

//カウントダウンを停止する
void stopTimer(){
  time = remainTime;         //設定時間に残り時間を代入する
}
//タイマーを更新する
void updateTimer(){
  remainTime = time + startTime - millis(); //残り時間を更新する
}
```

Actionタブ②

```
//アラームを鳴らす
void startAlarm(){
  startBuzzer();            //ブザーを鳴らす
  lightOn();                //LEDを点灯する
}

//アラームを止める
void stopAlarm(){
  stopBuzzer();             //ブザーを止める
  lightOff();               //LEDを消灯する
}

//時間と状態を表示する
void displayTimer(long ms, enum State s){

  int minute = int((ms/1000L)/60L);   //分を求める
  int second = int((ms/1000L)%60L);   //秒を求める
  int msec   = int(ms % 1000L);       //ミリ秒を求める
```

Actionタブ③

```
//表示する時間（文字列）を作成する
String str1 = "";
if(minute >= 0 && minute < 10){ //分が1桁の場合
  str1 += String("0");          //10の位に0を追加
}
str1 += String(minute);         //分を文字列str1に連結する
str1 += String(":");            //:を文字列str1に連結する

if(second >=0 && second < 10){  //秒が1桁の場合
  str1 += String("0");          //10の位に0を追加
}
str1 += String(second);         //秒を文字列str1に連結する
str1 += String(".");            //秒とミリ秒の間に .を追加

if(msec >= 0 && msec < 100){    //ミリ秒が2桁以下の場合
  str1 += String("0");          //100の位に0を追加
}
if(msec >=0 && msec < 10){      //ミリ秒が1桁以下の場合
  str1 += String("0");          //10の位に0を追加
}
str1 += String(msec);           //ミリ秒を文字列str1に連結する
```

Actionタブ④

```
  //表示する状態（文字列）を作成する
  String str2 = "";
  //状態を文字列str2に追加する
  if(s == STATE_INIT){
    str2 = String("STATE_INIT");
  } else if(s == STATE_SETTING){
    str2 = String("STATE_SETTING");
  } else if(s == STATE_COUNTDOWN){
    str2 = String("STATE_COUNTDOWN");
  } else if(s == STATE_PAUSE){
    str2 = String("STATE_PAUSE");
  } else if(s == STATE_ALARM){
    str2 = String("STATE_ALARM");
  } else {
    str2 = String("UNKNOWN");
  }

  //計測時間を1行目に表示、状態を2行目に表示
  displayString(str1, str2);
}
```

Deviceタブ①

```
 //Deviceタブ
#include <LiquidCrystal.h>
LiquidCrystal lcd(5, 6, 7, 8, 9, 10, 11);
const int SW1Pin = 2;   //StartStopボタン
const int SW2Pin = 4;   //分ボタン
const int LEDPin = 13;  //LED
const int SPKPin = 3;   //ブザー

//デバイスの初期設定を行う
void setupDevice(){
  pinMode(SW1Pin, INPUT);   //SW1Pinを入力設定
  pinMode(SW2Pin, INPUT);   //SW2Pinを入力設定
  pinMode(LEDPin, OUTPUT);  //LEDPinを出力設定
  lcd.begin(16, 2);         //LCDの設定
  lcd.clear();              //画面消去
  Serial.begin(9600);       //シリアルモニタ開始
}

//LCDに文字列を表示する
void displayString(String str1, String str2){
  lcd.clear();          //画面消去
```

Deviceタブ②

```
  lcd.setCursor(0, 0);  //上段左端に移動
  lcd.print(str1);      //文字列str1を表示
  lcd.setCursor(0, 1);  //下段左端に移動
  lcd.print(str2);      //文字列str2を表示
}

//スイッチを押したかどうか
boolean SWIsPressed(int pin){
  if(digitalRead(pin) == HIGH){ //pin番のスイッチが押された場合
    return true;                //trueを返す
  } else {                      //それ以外
    return false;               //falseを返す
  }
}

//ブザーを鳴らす
void startBuzzer(){
  tone(SPKPin, 880);  //トーン音を880Hzで鳴らす
}
```

Deviceタブ③

```
//ブザーを止める
void stopBuzzer(){
  noTone(SPKPin);           //トーン音を止める
}

//LEDを点灯する
void lightOn(){
  digitalWrite(LEDPin, HIGH); //LEDを点灯する
}

//LEDを消灯する
void lightOff(){
  digitalWrite(LEDPin, LOW);  //LEDを消灯する
}
```

Eventタブ①

```
//Eventタブ

//分ボタンを押したかどうか
boolean MinuteButtonIsPressed(){
  static boolean val_old = false;     //1回前の測定値を入れる変数
  boolean val = SWIsPressed(SW2Pin);  //現在の測定値
  boolean ret = false;                //戻り値を入れる変数

  if(val_old == false && val == true){  //OFFからONに変わったとき
    ret = true;
  } else {
    ret = false;
  }
  val_old = val;  //測定値更新
  return ret;
}

//StartStopボタンを押したかどうか
boolean StartStopButtonIsPressed(){
  static boolean val_old = false;     //1回前の測定値を入れる変数
  boolean val = SWIsPressed(SW1Pin);  //現在の測定値
```

Eventタブ②

```
  boolean ret = false;                    //戻り値を入れる変数

  if(val_old == false && val == true){  //OFFからONに変わったとき
    ret = true;
  } else {
    ret = false;
  }
  val_old = val;  //測定値更新
  return ret;
}

//残り時間が0になったかどうか
boolean TimeIsUp(){
  if(remainTime <= 0L){     //残り時間が0以下になる場合
    return true;
  } else {
    return false;
  }
}
```

Eventタブ③

```
//ここに課題プログラムで関数を追加

//イベント発生を確認する
enum Event receiveEvent(){
  //イベントが発生した場合、それ対応した列挙定数を返す

  //ここに課題プログラムで新しい条件を追加、以下の条件を一部修正
  if(MinuteButtonIsPressed()){
    return EVENT_PUSH_MINUTE_BUTTON;
  } else if(StartStopButtonIsPressed()){
    return EVENT_PUSH_START_STOP_BUTTON;
  } else if(TimeIsUp()){
    return EVENT_TIME_IS_UP;
  } else {
    return EVENT_NONE;
  }
}
```

Stateタブ①

```
//Stateタブ
//イベント発生時に対応した処理に割り振る関数
enum State handleEvent(enum State s, enum Event e) {
  enum State ret = s;
  switch (s) {
    case STATE_INIT:                    //初期状態の場合
      //必ず実行する処理
      time = 0L;                        //設定時間を初期化する
      startTime = 0L;                   //カウントダウン開始時間を初期化する
      remainTime = 0L;                  //残り時間を初期化する
      displayTimer(time, s);            //設定時間を表示する
      //イベント発生時の処理
      //ここに課題プログラムで条件を追加、修正
      if (e == EVENT_PUSH_MINUTE_BUTTON) {     //分ボタンを押した場合
        addOneMinute();                        //設定時間を1分増加
        ret = STATE_SETTING;                   //時間設定中に遷移
      } else if (e == EVENT_PUSH_START_STOP_BUTTON) { //StartStopボタンを押した場合
        //無視
      } else if (e == EVENT_TIME_IS_UP) { //残り時間が0になる場合
        //無視
      }
      break;
```

Stateタブ②

```
case STATE_SETTING:                 //時間設定中の場合
  //必ず実行する処理
  displayTimer(time, s);            //設定時間を表示する
  //イベント発生時の処理
  //ここに課題プログラムで条件を追加、修正
  if (e == EVENT_PUSH_MINUTE_BUTTON) {  //分ボタンを押した場合
    addOneMinute();                 //設定時間を1分増加
    ret = STATE_SETTING;            //時間設定中に遷移
  } else if (e == EVENT_PUSH_START_STOP_BUTTON) { //StartStopボタンを押した場合
    startTimer();                   //カウントダウンを開始する
    ret = STATE_COUNTDOWN;          //カウントダウン中に遷移する
  } else if (e == EVENT_TIME_IS_UP) { //残り時間が0になる場合
    //無視
  }
  break;

case STATE_COUNTDOWN:               //カウントダウン中の場合
  //必ず実行する処理
  updateTimer();                    //タイマーを更新する
  displayTimer(remainTime, s);      //残り時間を表示する
```

Stateタブ③

```
    //イベント発生時の処理
    //ここに課題プログラムで条件を追加、修正
    if (e == EVENT_PUSH_MINUTE_BUTTON) {  //分ボタンを押した場合
      //無視
    } else if (e == EVENT_PUSH_START_STOP_BUTTON) {//StartStopボタンを押した場合
      stopTimer();                    //カウントダウンを停止する
      ret = STATE_PAUSE;              //一時停止中に遷移する
    } else if (e == EVENT_TIME_IS_UP) { //残り時間が0になる場合
      startAlarm();                   //アラームを開始する
      ret = STATE_ALARM;              //アラーム鳴動中に遷移する
    }
    break;

  case STATE_PAUSE: //一時停止中の場合
    //必ず実行する処理
    displayTimer(time, s);            //設定時間を表示する
    //イベント発生時の処理
    //ここに課題プログラムで条件を追加、修正
    if (e == EVENT_PUSH_MINUTE_BUTTON) {  //分ボタンを押した場合
      addOneMinute();                 //設定時間を1分増加
      ret = STATE_SETTING;            //時間設定中に遷移する
```

Stateタブ④

```
    } else if (e == EVENT_PUSH_START_STOP_BUTTON) { //StartStopボタンを押した場合
      startTimer();                    //カウントダウンを開始する
      ret = STATE_COUNTDOWN;           //カウントダウン中に遷移する
    } else if (e == EVENT_TIME_IS_UP) { //残り時間が0になる場合
      //不可
    }
    break;

  case STATE_ALARM: //アラーム鳴動中の場合
    //必ず実行する処理
    remainTime = 0L;                   //残り時間を初期化する
    displayTimer(remainTime, s);       //残り時間を表示する
    startAlarm();                      //アラームを開始する
    //イベント発生時の処理
    //ここに課題プログラムで条件を追加、修正
    if (e == EVENT_PUSH_MINUTE_BUTTON) {  //分ボタンを押した場合
      //無視
    } else if (e == EVENT_PUSH_START_STOP_BUTTON) { //StartStopボタンを押した場合
      stopAlarm();                     //アラームを停止する
      ret = STATE_SETTING;             //時間設定中に遷移する
```

一部修正

Stateタブ⑤

```
      } else if (e == EVENT_TIME_IS_UP) { //残り時間が0になる場合
        //無視
      }
      break;

    default:  //それ以外
      ret = s;
      break;
  }
  return ret;
}
```

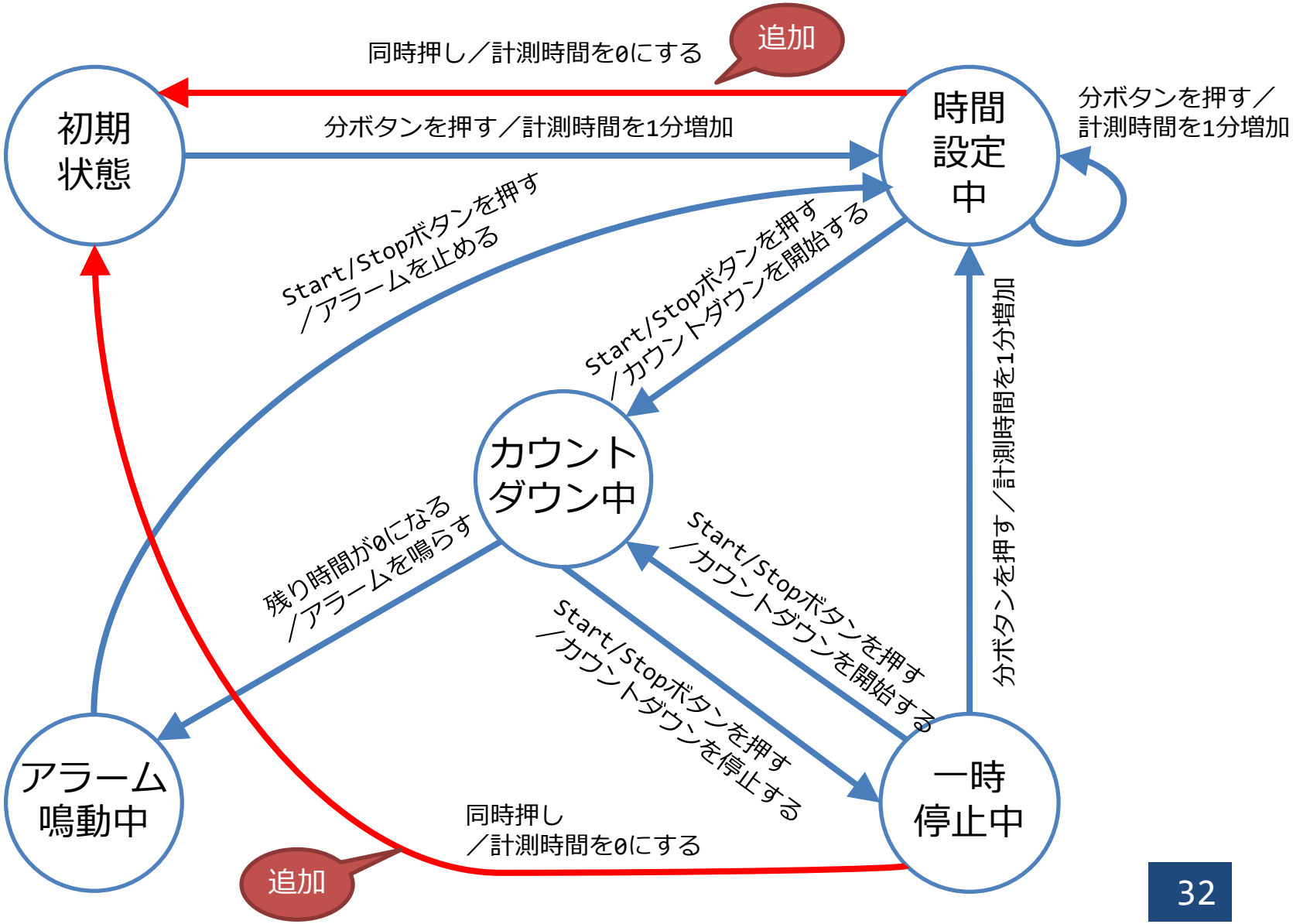
課題
仕様変更②

仕様変更①に加えて

分ボタン・Start/Stopボタン

同時押しで初期状態にする

仕様変更② 状態遷移図の変更



仕様変更②　状態遷移表の変更

- 時間設定中、一時停止中に分ボタンとStart/Stop ボタン同時押しで、設定時間を0にして初期状態に戻る

	初期状態	時間設定中	カウントダウン中	一時停止中	アラーム鳴動中
分ボタンを押す	計測時間を1分増加 時間設定中	計測時間を1分増加 時間設定中	／（無視）	計測時間を1分増加 時間設定中	／（無視）
Start/Stopボタンを押す	／（無視）	カウントダウンを開始する カウントダウン中	カウントダウンを停止する 一時停止中	カウントダウンを開始する カウントダウン中	アラームを止める 時間設定中
残り時間が0になる	／（無視）	／（無視）	アラームを鳴らす アラーム鳴動中	×（不可）	／（無視）
同時押し	**／（無視）**	**設定時間を0にする** **初期状態**	**／（無視）**	**設定時間を0にする** **初期状態**	**／（無視）**

追加　追加　追加　追加　追加　追加

プログラムの変更点

- 同時押しのイベントを表す列挙定数EVENT_PUSH_BOTH_BUTTONを追加
- 同時押しのイベント関数BothButtonIsPressed()を追加
- receiveEvent()関数に同時押しイベント発生の条件を追加
- handleEvent()関数に同時押しイベント発生時のアクションと状態遷移を追加

変更：メインタブ イベント追加

```
//config.h
enum State {
  STATE_INIT,
  STATE_SETTING,
  STATE_COUNTDOWN,
  STATE_PAUSE,
  STATE_ALARM
} state;

enum Event {
  EVENT_NONE,
  EVENT_PUSH_BOTH_BUTTON,
  EVENT_PUSH_MINUTE_BUTTON,
  EVENT_PUSH_START_STOP_BUTTON,
  EVENT_TIME_IS_UP
} event;
```

追加

変更箇所
- 列挙体enum Event型に
  両ボタン同時押しを示す列挙定数
  EVENT_PUSH_BOTH_BUTTONを追加

変更：Eventタブ　関数の追加・修正

```
//Eventタブ
boolean MinuteButtonIsPressed(){
  // 変更なし（省略）
}
boolean StartStopButtonIsPressed(){
 // 変更なし（省略）
}
boolean TimeIsUp(){
 // 変更なし（省略）
}
//両ボタン同時押ししたか
boolean BothButtonIsPressed(){
  if(SWIsPressed(SW1Pin) && SWIsPressed(SW2Pin)){
    return true;
  } else {
    return false;
  }
}
```

追加

SW1とSW2をどちらも押した場合trueを返す
それ以外はfalseを返す

```
enum Event receiveEvent(){
  if(BothButtonIsPressed()){
    return EVENT_PUSH_BOTH_BUTTON;
  } else if(MinuteButtonIsPressed()){
    return EVENT_PUSH_MINUTE_BUTTON;
  } else if(StartStopButtonIsPressed()){
    return EVENT_PUSH_START_STOP_BUTTON;
  } else if(TimeIsUp()){
    return EVENT_TIME_IS_UP;
  } else {
    return EVENT_NONE;
  }
}
```

変更

変更箇所
- 両ボタン同時押しが起きたか確認する関数BothButtonIsPressed()追加
- 関数receiveEvent()に同時押しを返す処理を追加する変更

変更：Stateタブ switch文

課題

- 関数handleEvent()の時間設定中、一時停止中に同時押しが発生した場合、以下の処理を行うように変更する
  - 設定時間を0にする
  - 初期状態に遷移する

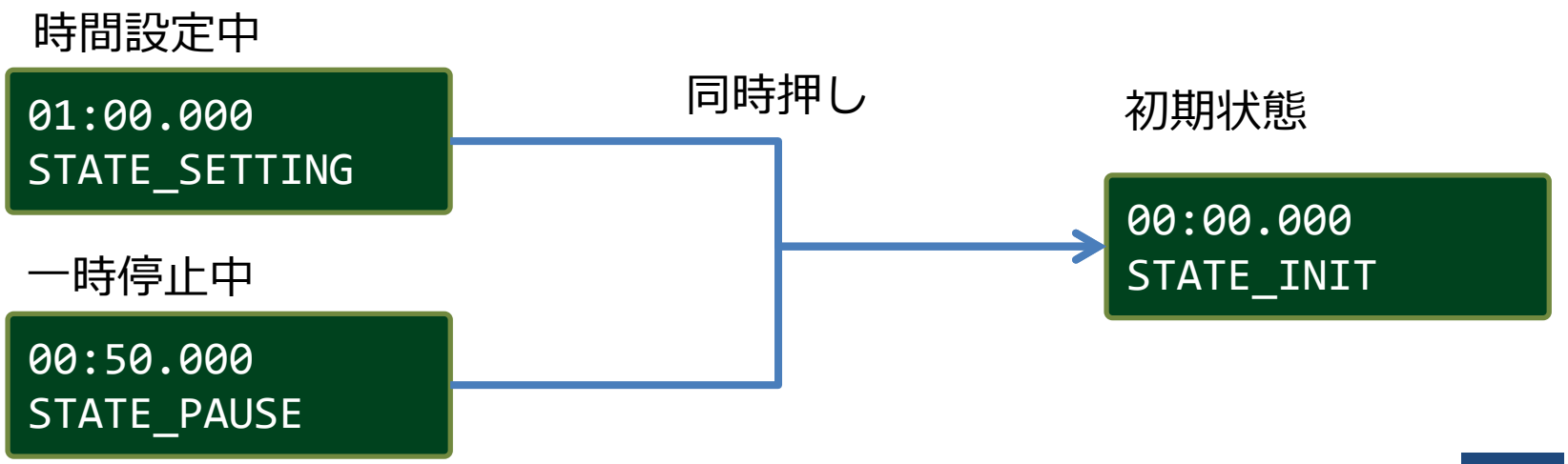
```
switch (s) {
   case STATE_INIT:  //初期状態
   //初期状態の処理
     break;
   case STATE_SETTING: //時間設定中
   //時間設定中の処理
     break;
   case STATE_COUNTDOWN: //カウントダウン中
   //カウントダウン中
     break;
   case STATE_PAUSE: //一時停止中
   //一時停止中の処理
     break;
   case STATE_ALARM: //アラーム鳴動中
   //アラーム鳴動中の処理
     break;
   default:  //それ以外
   //それ以外の処理
     break;
 }
```

変更

変更

動作確認

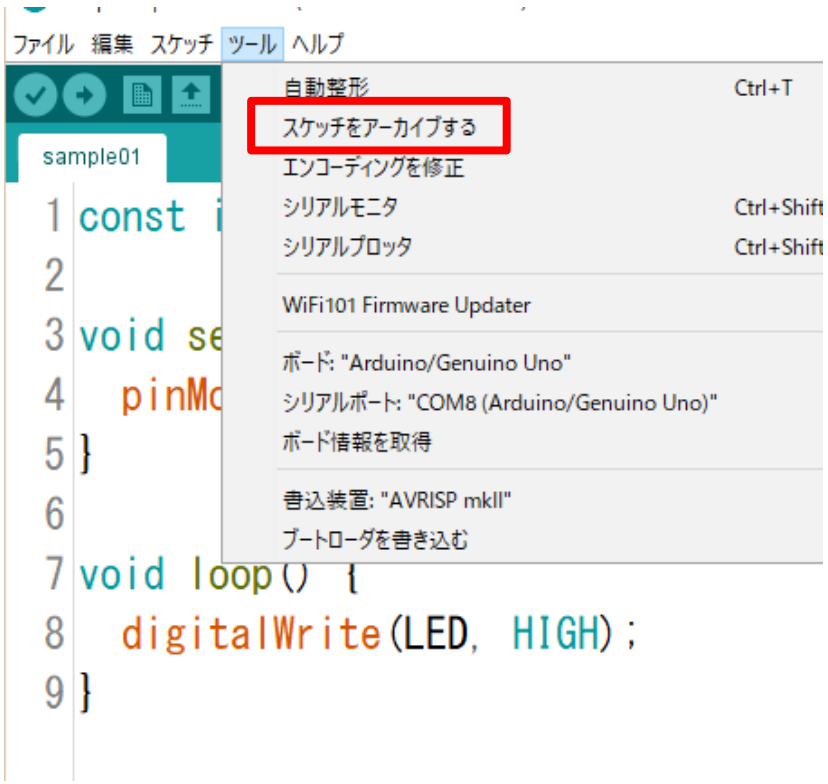
- 時間設定中・一時停止中で、同時押しした場合時間0、初期状態に遷移することを確認
- 同時押しのコツ
  - 一方のボタンを押したまま、他方のボタンを押す



課題の提出方法

プログラムをアーカイブする

- ツール ->
  スケッチをアーカイブ



- 圧縮ファイル名を指定

課題 提出先・期限

- 圧縮ファイルをメールに添付して提出する
  - 課題プログラム KT2018_05
  - 提出前に動作確認してから提出すること
- 提出先
  - haship@aomori-u.ac.jp
  - 件名：マイコンシステム15週
  - 本文に学籍番号、氏名を書く
- 提出期限
  - 2/4 13:00まで